STAREAST 2026 · ORLANDO · APRIL 29

Test and Measure the Output of Your AI Agents

A FIELD REPORT FROM THE FRONTIER OF NON-DETERMINISTIC QUALITY

Mikhail Nekorystnov

HOOK

You cannot fix what you do not know

WELCOME TO TESTING SOFTWARE THAT IS RIGHT ON TUESDAY AND WRONG ON WEDNESDAY — WITH THE SAME CODE.

About me — Mikhail (Misha) Nekorystnov

BUILDER · TROUBLESHOOTER · AI TRANSFORMATION LEAD

- I build and operate AI systems where mistakes cost money — literally.
- Most recent: healthcare reimbursement AI at human-grade error rates (**<0.5%**).
- 20+ years across engineering, product, and executive leadership.
- I am here because QA is evolving, not disappearing. I want to show you where.

I strive to make complex technology simpler. I work across engineering, product, operations, and business. Lately, my work has focused on AI, automation, and making it reliable.

Why this talk

Applications built on LLMs don't break the way applications used to break.

- Output depends on **model, system prompt, user prompt, context, tools** — all variables.
- "Same input, same output" is no longer true.
- Traditional test assertions miss **semantic** failures entirely.
- The question is not *did it run?* — it is *was the answer good enough, and how do you know?*

What you'll leave with

1. A **map** of the LLM eval landscape — who says what, what is settled, what isn't.
2. The **80/20** — what actually moves quality.
3. A **real story** — healthcare reimbursement evals, what we got wrong, what worked.
4. A **point of view** — where QA and PM roles are merging, and why that's good news.

PART 1

State of LLM Evals

WHAT THE FIELD HAS CONVERGED ON — AND WHERE IT STILL FIGHTS.

Evals 101 — the one distinction that matters

MODEL EVALS VS. PRODUCT EVALS

MODEL EVALS	PRODUCT EVALS
"What can this LLM do?"	"Does **our** app work for **our** users?"
MMLU, HumanEval, benchmarks	Task-specific, domain-specific
Useful for model selection	Required for quality measurement
Reusable across teams	Non-reusable — yours only

Most production failures live at the product layer. Benchmark scores won't catch them.

The industry has converged on a playbook

Across 50+ practitioner sources (Husain, Shankar, Iusztin, Reganti, Ng, Anthropic…):

1. **Look at real traces before writing any metric.** (Error analysis.)
2. **Build a small golden dataset.** 20–50 real examples is enough.
3. **Automate with an LLM-as-judge — then validate the judge.**
4. **Wire it into CI/CD and production monitoring.**
5. **Feed new failures back in.** It's a flywheel, not a checklist.

The 80/20 — highest ROI moves

IF YOU DO NOTHING ELSE, DO THESE.

- **Read 20 real traces.** By hand. This surfaces more bugs than any metric.
- **Use binary pass/fail.** Scales like 1–5 produce noise, not decisions.
- **Validate your LLM judge** against human labels — precision, recall, F1. Not only accuracy.
- **One principal domain expert** owns ground truth. Not a committee.
- **Treat evals as infrastructure**, not a pre-launch event.

Error analysis — the entry point

The universal first step. Low tech. High signal.

STEP	WHAT YOU DO
1. Sample	Pull 20–100 real traces
2. Open coding	Free-form note per trace: what went wrong
3. Cluster	LLM groups notes into named failure categories
4. Refine	Tighten categories until each is testable
5. Count	Sort by frequency. Top 3 become your eval targets.

LLM-as-Judge — and why the judge must itself be judged

> *A judge with a 100% pass rate is not a good judge. It is a broken one.*
>
> *— Hamel Husain*

- An LLM judge is a classifier with **unknown** performance until you measure it.
- Calibrate against 50–200 human-labeled examples.
- Report **precision, recall, F1** — never raw accuracy on skewed data.
- A judge that labels everything "pass" can show 90% accuracy and be useless.

Common pitfalls

- Chasing tools with **generic metrics** (BLEU, ROUGE, semantic similarity, etc.) because they're available.
- Writing the rubric **before** looking at outputs. *Criteria drift* — you can't.
- Outsourcing labeling to annotators **before** the team understands its own failures.
- Treating Likert 1–5 as signal. It isn't.
- Mocking the LLM in tests. The non-determinism **is** the thing you need to test.

Making evals less intimidating

You don't need a platform. You don't need a team. You don't need a PhD.

- A **spreadsheet** is a valid first eval tool.
- **20 traces** is a valid first dataset.
- **One afternoon** of reading output is a valid first sprint.
- The hard part is **reading**, not tooling. Start there.

> *"I couldn't define good feedback until I had seen bad feedback at scale." — Teresa Torres, a non-technical PM who built production evals.*

PART 2

My story

HEALTHCARE REIMBURSEMENT. HUNDREDS OF THOUSANDS OF DOCUMENTS. ERROR RATE <0.5%.

The use case

A healthcare reimbursement company. A human team processing hundreds of thousands of document submissions per year.

- Receipts, PDFs, medical bills, register prints — **nothing homogeneous**.
- Each submission: extract, classify expense, match to benefit period, match to IRS regulation.
- Humans achieved **<0.5% error rate.**
- The AI had to match it. People's money was at stake.

How we started

- Built an end-to-end pipeline: extract → normalize JSON → classify → validate against rules.
- Measured **one number**: overall error rate.
- Shipped a prototype. Ran it against real submissions.
- Confident. Clean architecture. Good logs.

It looked like it was working.

Early mistakes

- **Measured only aggregate error rate.** It hid which stage was failing.
- **No cut-off on extraction quality.** Bad OCR poisoned every downstream step.
- **Trusted the user's stated expense type.** We shouldn't have — verification was the whole job.
- **One eval for the whole pipeline.** When it failed, we couldn't tell *where*.
- **No golden dataset.** Every regression was re-discovered from scratch.

What we learned

1. **Evaluate each stage separately.** Extraction. Classification. Date match. Rule match.
2. **Gate on extraction quality.** If OCR confidence is low, stop — don't propagate noise.
3. **Build a golden set from real production documents.** Label by a domain expert, not by the AI.
4. **Binary pass/fail per stage.** Aggregate the stages, don't aggregate the judgment.
5. **The eval suite becomes the product spec.** Engineers hill-climb against it.

"Evals are the new PRDs."

— SHREYA SHANKAR & HAMEL HUSAIN, 2026

PART 3

The future of QA

NOT REPLACED. MERGED. EXPANDED.

QA and PM are converging

The old split:

- **PM** writes the PRD.
- **QA** verifies against the PRD.
- **Engineer** writes code between them.

The new shape for AI products:

- **The eval document is the PRD.**
- Whoever writes it, owns the product.
- QA + PM skill sets overlap: domain taste, failure-mode thinking, rubric design, measurement.

The case for the QA community

- **You already have** the mindset the field needs: adversarial thinking, edge-case discipline, repeatable process.
- **You already know** how to build test pyramids. Eval suites are the AI pyramid.
- **You already run** CI/CD gates. Production monitoring is the same loop, new signal.
- Domain expertise + error analysis = **product leverage**, not a supporting role.

Contrary to the common belief: **AI is not consuming QA jobs. It is expanding them.**

What to do Monday morning

- Pull **20 real outputs** from a shipping AI feature.
- Read them. Write a one-line note per trace.
- Cluster into **3–5 failure categories**.
- Pick the most frequent one. Write a binary pass/fail rubric.
- Ship a judge. Validate it against 50 human labels. Measure precision and recall.

You now have an eval flywheel. Everything else is scale.

THANK YOU

Questions?

MIKHAIL NEKORYSTNOV

STAREAST 2026 · ORLANDO · APRIL 29

LinkedIn: www.linkedin.com/in/mikhailnek/